



AULA 3

JavaScript

Nesta disciplina, está sendo desenvolvido o *site* do jogo *Whac-A-Mole*. Embora sua aparência tenha melhorado muito, até o momento todas as páginas são estáticas, sendo possível apenas navegar entre elas. A interação do usuário com as páginas é praticamente nula.

Nesta aula, será introduzido o uso de **JavaScript** no *site* para possibilitar essa interação.

Diferentemente de HTML, que é uma linguagem de descrição de leiaute, **JavaScript é uma linguagem de programação** que permite interagir com o usuário utilizando recursos até então fora do alcance de HTML com CSS, como tomada de decisão, repetição determinada ou indeterminada de comandos, entre outras.



Aprenda mais

Uma visão geral sobre a linguagem de programação JavaScript pode ser obtida no *site* da Fundação Mozilla. Lá, você também encontrará uma farta documentação.

JavaScript é uma linguagem de programação muito versátil, suportada por diversos navegadores *web*, para não dizer todos. Além de navegadores *web*, várias ferramentas suportam JavaScript, como servidores baseados em **Node.js** ou, até mesmo, banco de dados, como **MongoDB**.

Embora tenham estruturas parecidas, já tenham sido utilizadas para fins parecidos e tenham sido desenvolvidas

com base na linguagem C, não confunda Java com JavaScript.

Não use Java como uma forma abreviada, ou mais “íntima”, para se referir a JavaScript. Java e JavaScript são duas linguagens de programação com regras e objetivos bem distintos.



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Relembrar conceitos básicos de HTML e CSS.
- Reconhecer o papel do JavaScript no desenvolvimento *web*.
- Entender a manipulação básica do DOM.
- Implementar *scripts* em JavaScript para manipulação de eventos.
- Criar uma página interativa com HTML, CSS e JavaScript.

Que história é essa?

Um programa escrito em JavaScript pode ser executado em um navegador *web* na máquina do usuário do *site*, permitindo que o *script* interaja com elementos HTML e regras CSS das páginas.

Assim como HTML e CSS, código JavaScript pode ser desenvolvido utilizando qualquer editor de textos pleno. Escolha o editor ou ambiente (IDE – *integrated development environment*, ou ambiente de desenvolvimento integrado) que você considerar mais confortável. Existem várias opções gratuitas na internet.

Um bom IDE deve dar suporte a JavaScript sugerindo como completar o código escrito, analisando sua sintaxe e gramática.

Elementos de JavaScript

Antes de prosseguir, conheça algumas expressões que serão utilizadas nesta aula. A maioria delas não deve fazer muito sentido neste momento, mas todas serão explicadas com mais detalhes adiante.

Clique nos itens para saber mais:

Objeto	↓
Informações (dados) e funcionalidade juntos.	
Propriedade ou atributo	↓
Atributos (valores) que são associados a algum objeto.	

Método	↓
Uma ação que um objeto pode realizar.	
Evento	↓
Uma ação iniciada por um usuário ou pelo computador.	
Variável	↓
Um lugar para armazenar valores que podem variar ao longo da execução de um programa em um computador (propriedade está relacionada a objeto).	
Função	↓
Uma rotina ou procedimento que realiza uma ação (método está relacionado a objeto).	

Incluindo JavaScript

Existem três formas de incluir código JavaScript em uma página *web*.

Incluir o *script* junto com as *tags* HTML

Essa é a forma mais simples e somente utilizada em páginas pequenas e *sites* com poucas páginas.



Fique ligado

O uso de *script* junto com HTML dificulta o desenvolvimento de *sites* maiores, sendo uma prática não recomendada pelo paradigma MVC (*model-view-controller*).

No modelo MVC, os componentes de cada parte devem ficar isolados para permitir separação de responsabilidade, reutilização de código, desenvolvimento em equipe, entre outros.

O código a seguir apresenta uma página *web* com JavaScript embutido na *tag* HTML:

Interativo

Descrição do interativo

Entenda o que aconteceu. Com exceção da linha 11, o código deve ser bem familiar a você. É um código puramente HTML que descreve uma página com um botão e um parágrafo (sem texto).

Esse parágrafo foi criado com uma identificação (**lugar**). A linha 11 aplica à *tag* **<button>** um atributo que ainda não havia sido utilizado: **onclick**. Como o nome sugere, esse atributo irá adicionar um comportamento (evento) a essa *tag* associado ao clique do *mouse*. Ou seja, quando o usuário clicar no botão, o código JavaScript associado ao evento será executado.

Nesse comando JavaScript, o objeto **document**, que representa todo o documento HTML, incluindo CSS e JavaScript, é utilizado para realizar a ação (método) de pegar um elemento HTML pela sua identificação. A partir desse objeto HTML, agora no domínio JavaScript, pode-se incluir em seu contêiner, usando a propriedade **innerHTML**, uma mensagem.

Note que a mensagem está escrita entre aspas simples, para não confundir com as aspas duplas usadas para definir o valor do atributo **onclick**.

Declarar um bloco *script* na página

A segunda forma de incluir código JavaScript em uma página *web* é declarar um bloco *script* nela. Nesse caso, temos uma separação do código JavaScript (*controller*), do código HTML e do CSS (*view*). A página que foi montada ficou mais organizada. Porém, o código JavaScript incorporado nela não pode ser utilizado em outras páginas do *site*. Se você precisar do mesmo código em outra página, terá que escrevê-lo novamente, duplicando o trabalho. Além disso, se precisar modificar o código JavaScript por uma mudança qualquer de definição de algum comportamento, terá que fazer a mesma mudança em todas as páginas HTML que tiverem esse código.

O exemplo a seguir mostra uma página semelhante à do caso anterior:

<> código ▾

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <script>
7          document.getElementById('btnIniciar').addEventListener('click', mostraMensagem);
8          function mostraMensagem() {
9              document.getElementById('lugar').innerHTML = 'Alô Mundo!!!';
10         }
11     </script>
12     <title>Alô mundo JavaScript</title>
13 </head>
14 <body>
15     <h1>Exemplo de JavaScript no HEAD</h1>
16     <button id="btnIniciar">
17         Aperte aqui para começar a descobrir um Admirável Mundo Novo! XD
18     </button>
19     <p id="lugar"></p>
20 </body>
21 </html>
```



Nessa página, o código JavaScript foi colocado na seção **<head>** porque, teoricamente, seria o melhor lugar, uma vez que o código em si não faz parte do visual da página, apenas descreve seu comportamento.

Para associar o evento de clicar no botão a exibir a mensagem, foi necessário o uso de mais uma novidade: adicionar um *listener* (um “ouvidor”) de evento, ou seja, um trecho de código que fica responsável por tratar o evento assim que ele ocorre.

A linha 7 contém o código JavaScript que realiza essa tarefa. Novamente, é usado o objeto **document** para pegar um elemento HTML por meio de seu *id* (**btnIniciar**), mas dessa vez foi adicionado um *listener* de evento associado ao código da função **mostraMensagem**. A partir desse instante, sempre que houver um evento de clique no elemento identificado por **btnIniciar**, ou seja, o

botão, o código descrito na função **mostraMensagem** será executado.

Ao se tentar executar o código JavaScript clicando no botão, nada acontece. Uma ferramenta simples que pode ajuda a resolver esse problema é o *console* do desenvolvedor. Para acessar o *console*, digite **Ctrl + Shift + i** e clique na aba **Console**.

À primeira vista, a mensagem de erro que aparece não ajuda muito, mas surgirá tantas vezes na tela que ficará familiar. A mensagem informa que algum objeto não está sendo corretamente referenciado, seja porque não foi corretamente inicializado (ainda não foi feito isso) ou porque não existe (que é o caso).



Fique ligado

A explicação é que o navegador constrói a página *web* à medida que vai recebendo o código do servidor. O código JavaScript, como foi colocado na página primeiro que o objeto, foi executado antes de o objeto botão ser renderizado. Assim, a linha 7 tentou pegar um objeto que ainda não existia.

A resolução desse problema pode ser feita por meio de duas abordagens:

(a) associar a execução do código ao carregamento completo da página *web* (é uma boa solução, mas ainda é muito cedo para saber implementar); ou

(b) mover o código JavaScript para o final da página (como ele não será exibido, tecnicamente pode ficar em qualquer lugar).

Interativo

Descrição do interativo

Usar um arquivo dedicado ao *script*

Para obter uma separação total do código JavaScript do código HTML/CSS, pode-se usar um arquivo dedicado ao *script*. Embora não seja obrigatório, normalmente a extensão **.js** é utilizada no nome do arquivo.

Para criar um *link* entre o código HTML e o *script*, deve-se incluir na página *web* a tag **<script>** e especificar o arquivo usando o atributo **src**. O melhor lugar para incluir esse *link* ao arquivo com o *script* é a seção **<head>** da página, mas, por enquanto, será incluído no final, pelo mesmo motivo apresentado no exemplo anterior.

O código da nova página *web* ficou assim:

código

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Alô mundo JavaScript</title>
7 </head>
8 <body>
9   <h1>Exemplo de JavaScript no HEAD</h1>
10  <button id="btnIniciar">
11    Aperte aqui para começar a descobrir um Admirável Mundo Novo! XD
12  </button>
13  <p id="lugar"></p>
14 </body>
15 <script src="alo_mundo.js"></script>
16 </html>
```

Note que a tag **<script>**, para manter compatibilidade, continua sendo do tipo contêiner, mesmo não tendo conteúdo. No passado, o conteúdo era uma mensagem informando que o navegador utilizado não provia suporte a JavaScript.

O arquivo **alo_mundo.js** foi colocado no mesmo diretório do arquivo HTML. Isso não é necessário. O melhor seria criar um diretório para os arquivos *script* e informar o caminho no atributo de **src**. O conteúdo do arquivo com o *script* ficou assim:

código

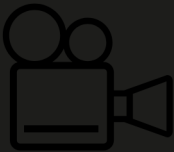
```
1 document.getElementById('btnIniciar').addEventListener('click', mostraMensagem);
2 function mostraMensagem() {
3   document.getElementById('lugar').innerHTML = 'Alô Mundo!!!';
4 }
```

Note a ausência da tag **<script>** e de qualquer outra tag HTML. O arquivo é puramente JavaScript.

O efeito é o mesmo nos três casos.

Construção do código

Neste tópico, será visto o que são funções, funções anônimas e seu funcionamento. Antes, porém, assista à videoaula a seguir:



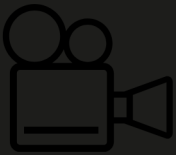
Constantes e variáveis

Assista, agora, ao vídeo sobre constantes e variáveis para entender melhor a construção do código.



Funções

Antes de começar a falar sobre funções, é importante que você tenha o conceito de operadores e expressões.



Operadores e expressões

Assista ao vídeo para conhecer os operadores e expressões.



A maior parte do código JavaScript é escrito dentro de funções. Uma função é um trecho de código que somente será executado quando referenciado pelo seu nome seguido de parênteses ou quando acontecer o evento a que está associado.

Basta escrever uma vez a função e chamar quantas vezes forem necessárias.

De uma maneira genérica, uma função se parece com o código a seguir:

```
function nomeDaFuncao(parametros, separados, por, vírgula) {  
  // comentário  
  // corpo da função  
  // código JavaScript  
  return; // ou return valor ==> ambos opcionais  
}
```

Exemplo de função

Por exemplo, a função a seguir recebe como parâmetro o valor de um ponto no eixo cartesiano (X,Y) e calcula sua distância em relação à origem, ou seja, ao ponto (0,0). A distância é a raiz quadrada da soma do quadrado de cada coordenada. A função implementada se chama **calculaDistancia**, recebe as coordenadas **x** e **y** como parâmetro de entrada e retorna a distância do ponto à origem do plano.

```
11 function calculaDistancia(x,y) {  
12   var distancia = Math.sqrt(x*x + y*y);  
13   return distancia;  
14 }
```

O seguinte trecho de código cria dois pontos no eixo cartesiano e informa qual está mais longe por meio do *console* do navegador.

Para visualizar o resultado, digite **Ctrl + Shift + i** e clique na aba **Console**.

```
16 // primeiro ponto
17 var x1 = 3;
18 var y1 = 7;
19
20 // segundo ponto
21 var x2 = 5;
22 var y2 = 2;
23
24 // calculo das distâncias
25 var dist1 = calculaDistancia(x1, y1);
26 var dist2 = calculaDistancia(x2, y2);
27
28 // exibe a informação
29 if(dist1 > dist2) {
30   console.log('O 1o ponto está mais longe');
31 } else if(dist1 < dist2) {
32   console.log('O 2o ponto está mais longe');
33 } else {
34   console.log('Os pontos estão equidistantes');
35 }
```

Note que todos os comandos JavaScript terminam com o símbolo de ponto e vírgula.

Funcionamento

As linhas 16, 20, 24 e 28 contêm comentários, ou seja, texto que não será utilizado pelo motor JavaScript do navegador, servindo apenas para documentação do código para humanos.

O primeiro e o segundo ponto são criados nas linhas 17, 18, 21 e 22, criando e inicializando as variáveis **x1**, **y1** e **x2**, **y2**, respectivamente.

Na linha 25, tem-se a chamada da função **calculaDistancia** para calcular a distância do primeiro ponto. Nesse momento, o primeiro valor informado dentro dos parênteses é atribuído à primeira variável declarada como parâmetro da função. O segundo valor informado dentro dos parênteses é atribuído à segunda variável declarada como parâmetro da função, e assim por diante. Terminada a fase de atribuições de valores, o controle é transferido para a função que é executada até o final, ou seja, o código entre o abre e fecha chaves, ou até encontrar o comando **return**. Em ambos os casos, o controle volta para o ponto de chamada da função.

Se for informado um valor junto ao comando **return**, esse valor é retornado pela função e podemos imaginar que, nesse ponto, a função assume esse valor, ou seja, o valor retornado é atribuído à variável **dist1**.

O mesmo acontece na linha 26.

Nas linhas 26, 31 e 33, verifica-se a relação de distância dos pontos e informa-se o resultado no *console* pelas linhas 30, 32 e 34.

Funções anônimas

Opcionalmente, podem-se criar funções anônimas, ou seja, sem nome. Essas funções devem ser atribuídas a atributos ou variáveis durante sua construção.

Normalmente, criam-se funções anônimas em *callbacks* (tratamento de eventos, por exemplo) ou quando se quer evitar poluição no escopo global, uma vez que uma função anônima não tem um nome global (ela não tem nome). Em algumas situações, uma função anônima também torna o código mais claro, melhorando sua legibilidade e concisão.

O exemplo a seguir mostra uma função anônima sendo atribuída à propriedade **onclick** de um elemento HTML identificado por **idButton**:

```
11 document.getElementById("idButton").onclick = function() {
12 |   alert("O botão foi clicado!");
13 | };
```

Mais tarde, veremos como usar o método **addEventListener** para fazer a mesma coisa.